Experiment 13

Name: N. PARNITA **Course Code: MLA0305**

Regno: 192525322 **Slot: B**

Title: Advanced Policy Optimization using PPO, TRPO and DDPG

Software Required:

- Stable-Baselines3
- TensorFlow
- Gymnasium

AIM

To implement advanced policy optimization algorithms **PPO, TRPO and DDPG** and compare their Reinforcement Learning performance.

THEORY

PPO (Proximal Policy Optimization) improves the policy while restricting excessively large updates. **TRPO (Trust Region Policy Optimization)** constrains policy updates to maintain stable learning. **DDPG (Deep Deterministic Policy Gradient)** is an actor-critic algorithm designed for continuous action spaces.

PROCEDURE

1. Set up the Python environment with the required Reinforcement Learning libraries.
2. Import Stable-Baselines3, TensorFlow and Gymnasium.
3. Create suitable environments for the selected algorithms.
4. Configure the PPO algorithm.
5. Configure the TRPO algorithm.
6. Configure the DDPG algorithm.
7. Train each algorithm for the specified number of timesteps.
8. Evaluate the trained agents using their obtained rewards.
9. Record the performance of PPO, TRPO and DDPG.
10. Compare the performance of the three algorithms using a graph.

CODE:

```
# EXPERIMENT 13
# PPO vs TRPO vs DDPG
# Advanced Policy Optimization

!pip -q install stable-baselines3 sb3-contrib gymnasium

import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

from stable_baselines3 import PPO, DDPG
from sb3_contrib import TRPO

# ====================================================
# 1. CREATE ENVIRONMENTS
# ====================================================

# PPO and TRPO can work with discrete CartPole
env_ppo = gym.make("CartPole-v1")
env_trpo = gym.make("CartPole-v1")

# DDPG requires a continuous action space
env_ddpg = gym.make("Pendulum-v1")

# ====================================================
# 2. CREATE MODELS
# ====================================================

ppo_model = PPO(
    "MlpPolicy",
    env_ppo,
    verbose=0,
    learning_rate=0.0003
)

trpo_model = TRPO(
    "MlpPolicy",
    env_trpo,
```

```
    verbose=0,
    learning_rate=0.0003
)

ddpg_model = DDPG(
    "MlpPolicy",
    env_ddpg,
    verbose=0,
    learning_rate=0.001
)

# ======================================================
# 3. TRAIN MODELS
# ======================================================

TIMESTEPS = 3000

print("Training PPO...")
ppo_model.learn(total_timesteps=TIMESTEPS)
print("PPO training completed.")

print("\nTraining TRPO...")
trpo_model.learn(total_timesteps=TIMESTEPS)
print("TRPO training completed.")

print("\nTraining DDPG...")
ddpg_model.learn(total_timesteps=TIMESTEPS)
print("DDPG training completed.")

# ======================================================
# 4. EVALUATION FUNCTION
# ======================================================

def evaluate_agent(model, env, episodes=10):

    rewards = []

    for _ in range(episodes):

        obs, info = env.reset()
        total_reward = 0
        done = False
```

```
        while not done:

            action, _ = model.predict(
                obs,
                deterministic=True
            )

            obs, reward, terminated, truncated, info = env.step(
                action
            )

            done = terminated or truncated
            total_reward += reward

        rewards.append(total_reward)

    return rewards


# ======================================================
# 5. EVALUATE
# ======================================================

print("\nEvaluating PPO...")
ppo_rewards = evaluate_agent(
    ppo_model,
    env_ppo
)

print("Evaluating TRPO...")
trpo_rewards = evaluate_agent(
    trpo_model,
    env_trpo
)

print("Evaluating DDPG...")
ddpg_rewards = evaluate_agent(
    ddpg_model,
    env_ddpg
)
```

```python
# ======================================================
# 6. DISPLAY RESULTS
# ======================================================

print("\n========================================")
print("PERFORMANCE SUMMARY")
print("========================================")

print(
    "PPO Average Reward  :",
    round(np.mean(ppo_rewards), 2)
)

print(
    "TRPO Average Reward :",
    round(np.mean(trpo_rewards), 2)
)

print(
    "DDPG Average Reward :",
    round(np.mean(ddpg_rewards), 2)
)

# ======================================================
# 7. COMPARISON GRAPH
# ======================================================

methods = [
    "PPO",
    "TRPO",
    "DDPG"
]

average_rewards = [
    np.mean(ppo_rewards),
    np.mean(trpo_rewards),
    np.mean(ddpg_rewards)
]

plt.figure(figsize=(9, 5))

bars = plt.bar(
    methods,
```

```
    average_rewards
)

plt.xlabel("Algorithm")
plt.ylabel("Average Evaluation Reward")

plt.title(
    "Performance Comparison of PPO, TRPO and DDPG"
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.6
)

# Display values above bars
for bar, value in zip(
    bars,
    average_rewards
):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height(),
        f"{value:.2f}",
        ha="center",
        va="bottom"
    )

plt.tight_layout()
plt.show()

# ======================================================
# 8. CLOSE ENVIRONMENTS
# ======================================================

env_ppo.close()
env_trpo.close()
env_ddpg.close()

print("\nExperiment 13 completed successfully!")
```

OUTPUT:

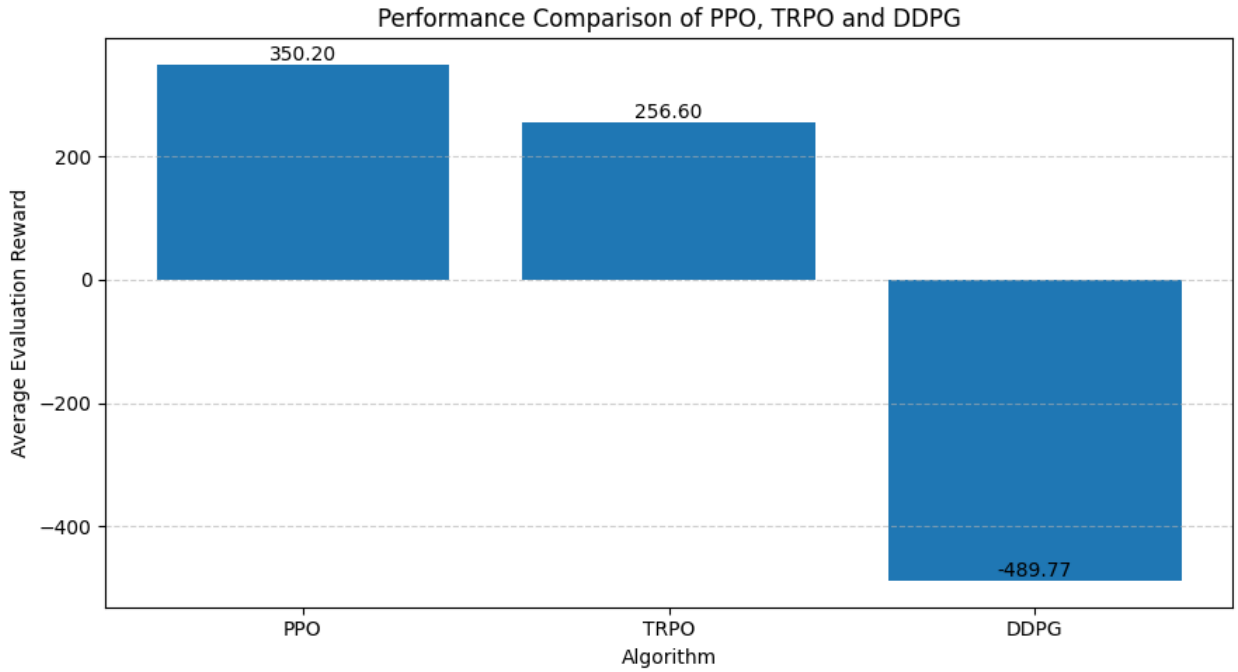
```
PPO training completed.

Training TRPO...
TRPO training completed.

Training DDPG...
DDPG training completed.

Evaluating PPO...
Evaluating TRPO...
Evaluating DDPG...

========================================
PERFORMANCE SUMMARY
========================================
PPO Average Reward  : 350.2
TRPO Average Reward : 256.6
DDPG Average Reward : -489.77
```




```
Experiment 13 completed successfully!
```

RESULT:

PPO, TRPO and DDPG were successfully implemented and their policy optimization performance was compared.